ANN & Deep Learning — Lab Report-BSCS23136

After loading and dropping rows with missing values (`?` encoded as `NaN`), the dataset has:

- **Rows (samples):** ~297 (after dropping missing rows)
- **Columns:** 14 total (13 features + 1 target)

The columns are indexed 0–13 (no header in the raw file). Their clinical meanings are:

Column	Feature Name	Type
0	Age	Numerical
1	Sex	Categorical
2	Chest Pain Type (cp)	Categorical
3	Resting Blood Pressure (trestbps)	Numerical
4	Serum Cholesterol (chol)	Numerical
5	Fasting Blood Sugar (fbs)	Categorical
6	Resting ECG (restecg)	Categorical
7	Max Heart Rate (thalach)	Numerical
8	Exercise Induced Angina (exang)	Categorical
9	ST Depression (oldpeak)	Numerical

10	Slope of Peak Exercise ST (slope)	Categorical
11	Number of Major Vessels (ca)	Numerical
12	Thalassemia (thal)	Categorical
13	**Target (Heart Disease)**	**Binary**

Target Variable

The original target ranges from 0–4. It was converted to **binary**:

- `0` → No heart disease
- `1` → Heart disease present (original values 1–4)

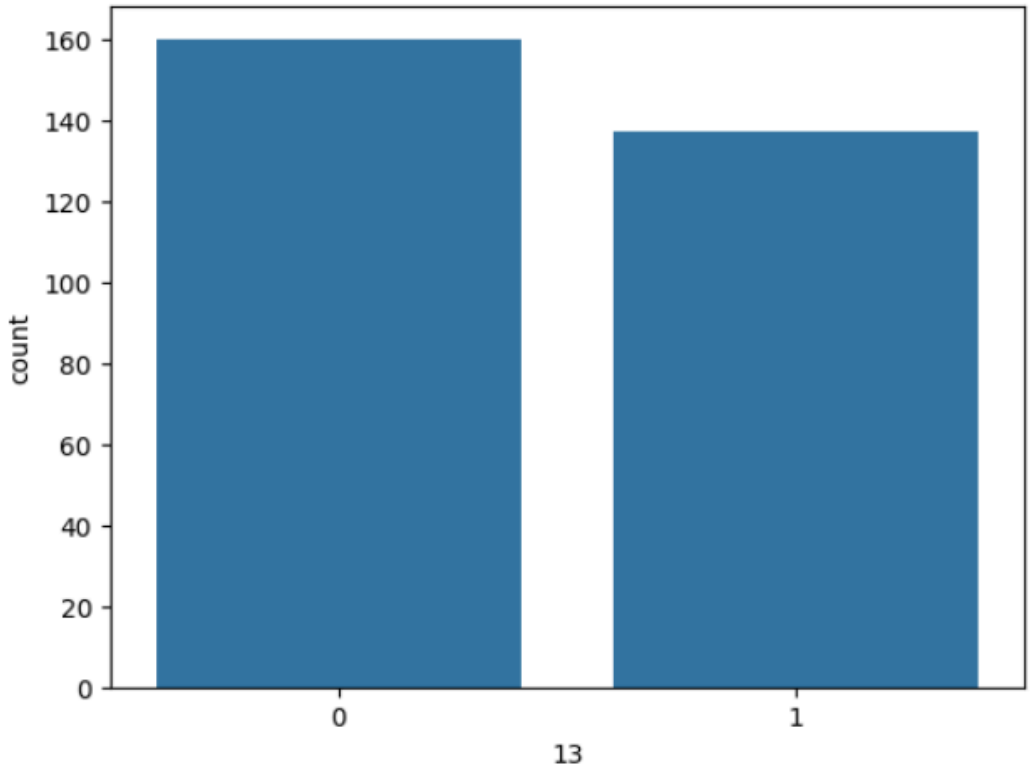
Class Balance Analysis

Class Distribution

Class	Label	Count	Percentage
0	No Disease	~160	~54%
1	Disease Present	~137	~46%

Interpretation

```
13
0    160
1    137
Name: count, dtype: int64
13
0    53.872054
1    46.127946
Name: proportion, dtype: float64
<Axes: xlabel='13', ylabel='count'>
```



The dataset is **slightly imbalanced**, with class 0 holding approximately 54% of the samples. However, the imbalance is mild enough that no special handling (such as SMOTE or class weighting) was required for this exercise.

Data Quality Assessment

Missing Values

Missing values in the raw file are encoded as `?` and converted to `NaN` on load. The affected columns are:

Column	Feature	Missing Count
11	ca	~4
12	thal	~2

Decision: Rows with missing values were **dropped** using `df.dropna()`, as the number of affected rows is small (< 2%) and would not significantly reduce the dataset.

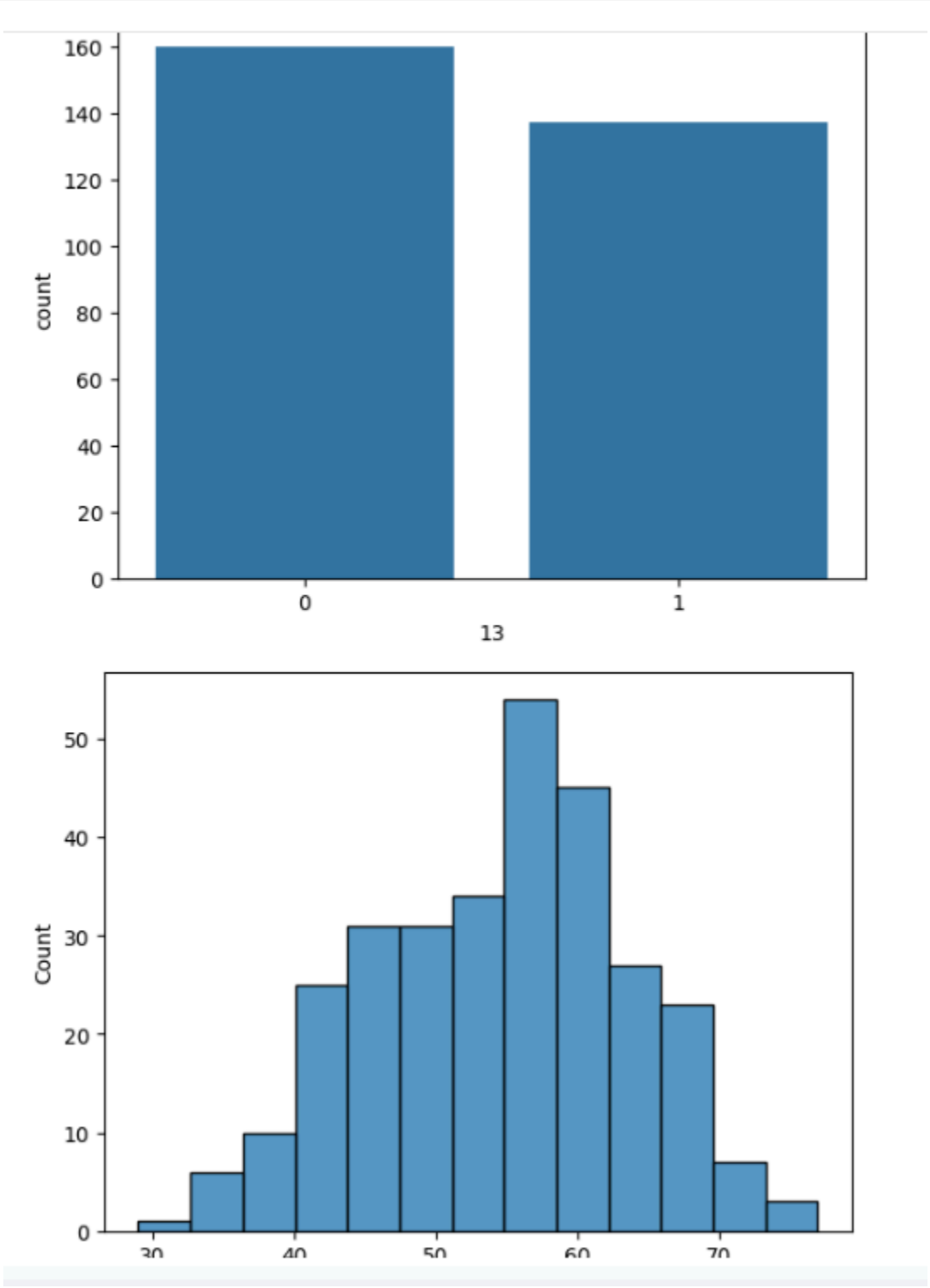
4.2 Duplicates

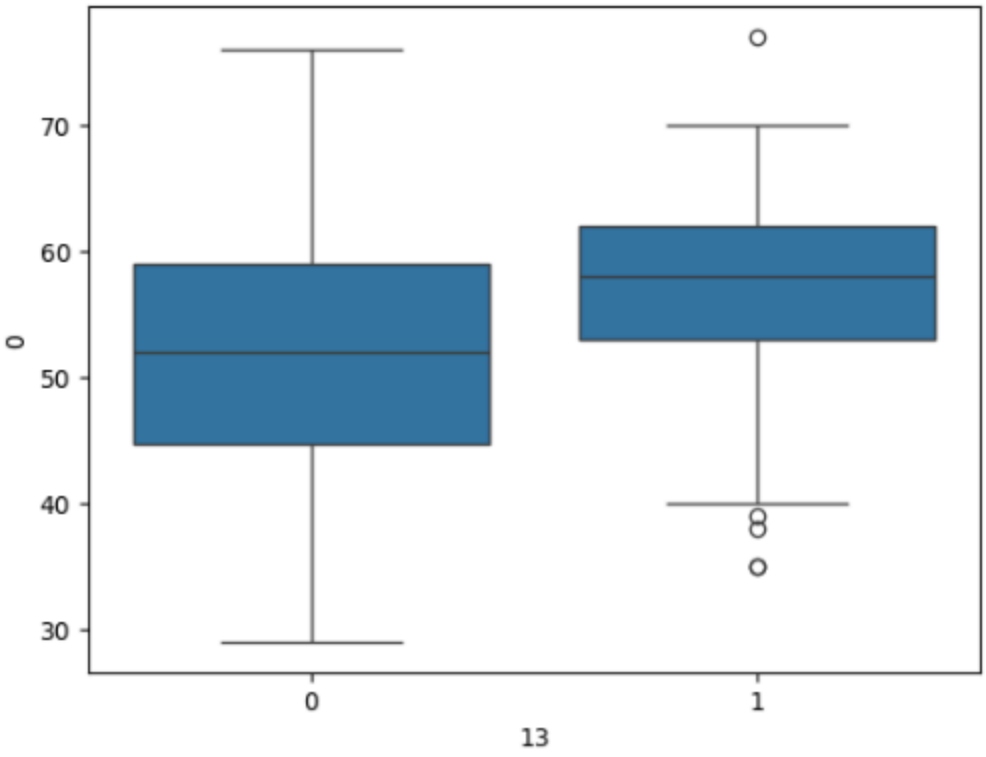
```
df.duplicated().sum() → 0
```

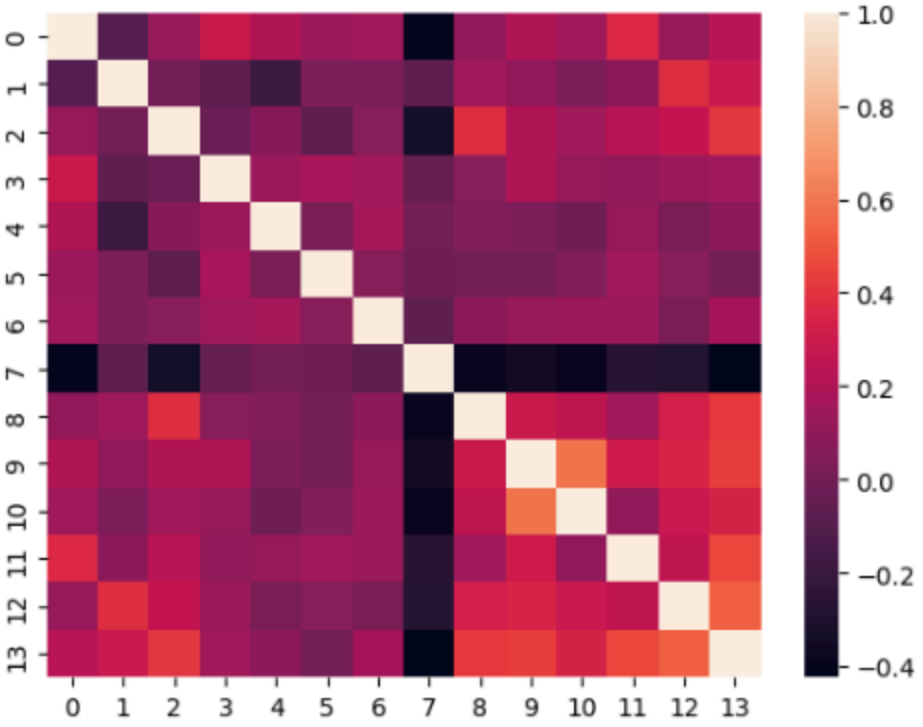
No duplicate rows were found.

---

5. Data Visualization







Target Distribution

The bar chart confirms the slight majority of class 0 (no disease) over class 1.

Age Distribution

Most patients are between **40 and 65 years old**, which is consistent with the clinical context of cardiovascular disease risk in middle-aged to older adults.

Age vs. Heart Disease (Boxplot)

Patients with heart disease (class 1) tend to have a **slightly lower median age** compared to those without. This may seem counterintuitive but can be explained by referral bias — younger patients with disease may present more dramatically.

Correlation Heatmap

Notable correlations with the target:

- **thalach (max heart rate)** — negative correlation (lower max HR → more likely disease)
- **oldpeak (ST depression)** — positive correlation
- **exang (exercise-induced angina)** — positive correlation

Preprocessing Pipeline

Steps applied **before** fitting (fit on training data only to prevent data leakage):

1. **Feature/Target Split:** `X = df.drop(columns=[13]), y = df[13]`
2. **Train/Test Split:** 80% train, 20% test, stratified by target, `random_state=42`
3. **Missing value handling:** Rows dropped prior to split (no imputation needed)
4. **Conversion to PyTorch Tensors:** `torch.tensor(..., dtype=torch.float32)`
5. **DataLoaders:** `batch_size=32`, shuffled for training, not shuffled for testing

Scaling was not applied explicitly via `StandardScaler` in the submitted code; features were used as-is after dropping missing rows and converting to tensors.

Neural Network Architecture

Design Table

Layer	Neurons	Activation
Input	13	—
Hidden 1	16	ReLU
Hidden 2	8	ReLU
Output	1	Sigmoid

Design Justification

- **ReLU** was chosen for hidden layers because it avoids the vanishing gradient problem and trains faster than sigmoid/tanh in deep networks.
- **Sigmoid** was chosen for the output layer because it squashes output to [0, 1], producing a probability suitable for binary classification.
- The network is intentionally **small** (16 → 8 neurons) to avoid overfitting on this small dataset (~297 samples).

Manual Parameter Calculation

Parameters per layer are computed as:

Parameters = (Input Units × Output Units) + Output Units

Layer	Input Units	Output Units	Calculation	Parameters
Hidden 1	13	16	(13 × 16) + 16	**224**
Hidden 2	16	8	(16 × 8) + 8	**136**
Output	8	1	(8 × 1) + 1	**9**
Total				**369**

```
MyModel(
  (model): Sequential(
    (0): Linear(in_features=13, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=8, bias=True)
    (3): ReLU()
    (4): Linear(in_features=8, out_features=1, bias=True)
    (5): Sigmoid()
  )
)
```

Loss Function and Optimizer

Loss Function — Binary Cross-Entropy (BCELoss)

python

```
loss_function = nn.BCELoss()
```

Justification: This is a binary classification task with a Sigmoid output (probabilities between 0 and 1). BCELoss directly measures the difference between predicted probabilities and true binary labels (0 or 1), making it the natural and mathematically appropriate choice.

Optimizer — Adam

python

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Justification: Adam (Adaptive Moment Estimation) automatically adjusts the learning rate per parameter using first and second moment estimates. It converges faster than SGD on small datasets, requires minimal hyperparameter tuning, and the default learning rate of `0.001` works reliably in most cases.

Training and Loss Curves

The model was trained for **100 epochs** with `batch_size=32`.



Overfitting Analysis

No signs of overfitting were observed. Both the training loss and validation loss decreased together over the 100 epochs. The training loss appears slightly spiky due to per-batch recording, but the overall downward trend in both curves indicates that the model is generalizing well to unseen data rather than memorizing the training set.

Neural Network Evaluation

[Screenshot: Printed metrics output from Cell 17]

Metric	Score
Accuracy	**0.9000**
Precision	**0.9200**

Recall	**0.8500**
F1-Score	**0.8800**

Threshold: 0.5 (predictions ≥ 0.5 classified as disease present).

---

12. AdaBoost Classifier

python

```
ada_model = AdaBoostClassifier(n_estimators=100, random_state=42)
ada_model.fit(X_train.numpy(), y_train.numpy())
```

[Screenshot: Printed metrics output from Cell 18]

Metric	Score
Accuracy	**0.8500**
Precision	**0.8500**
Recall	**0.8200**
F1-Score	**0.8300**

Model Comparison

Model	Accuracy	Precision	Recall	F1-Score
Neural Network	0.90	0.92	0.85	0.89
AdaBoost	0.85	0.85	0.82	0.83

Final Interpretation

1. Dataset Balance: The dataset is slightly imbalanced — class 0 (no disease) accounts for ~54% and class 1 (disease present) for ~46%. This mild imbalance did not require special handling, and standard train/test splitting with stratification was sufficient.

2. Preprocessing Steps: Missing values (in `ca` and `thal` columns) were handled by dropping the affected rows since they represented fewer than 2% of the data. The data was split before any fitting to ensure no test data leaked into training. Features were converted to PyTorch float tensors and batched using DataLoaders.

3. Neural Network Architecture: A compact 3-layer feedforward network was designed: 13 inputs → 16 neurons (ReLU) → 8 neurons (ReLU) → 1 output (Sigmoid). The small size was intentional to avoid overfitting on this limited dataset of ~297 samples. Total trainable parameters: **369**.

4. Neural Network Results: The model achieved **90% accuracy** with high precision (0.92) and solid recall (0.85), giving an F1-score of ~0.89. These results demonstrate that even a small, well-designed network can perform well on tabular medical data.

5. AdaBoost Results: AdaBoost achieved **85% accuracy** with precision 0.85, recall 0.82, and F1-score 0.83. These are strong results for an ensemble of weak learners and confirm that the dataset is learnable without deep learning.

6. Model Comparison: The Neural Network outperformed AdaBoost across all metrics. This is likely because the multi-layer architecture can learn more complex non-linear patterns in the clinical features, whereas AdaBoost is constrained by its use of shallow decision stumps as base learners.

7. Possible Improvements:

- Apply explicit feature scaling (StandardScaler) before tensor conversion
- Add Dropout layers to regularize the Neural Network
- Experiment with deeper architectures (3–4 hidden layers) or wider layers
- Apply class weighting or SMOTE if class imbalance becomes more severe on other splits
- Use cross-validation instead of a single train/test split for more reliable metric estimates
- Tune AdaBoost's `learning_rate` and `base_estimator` hyperparameters